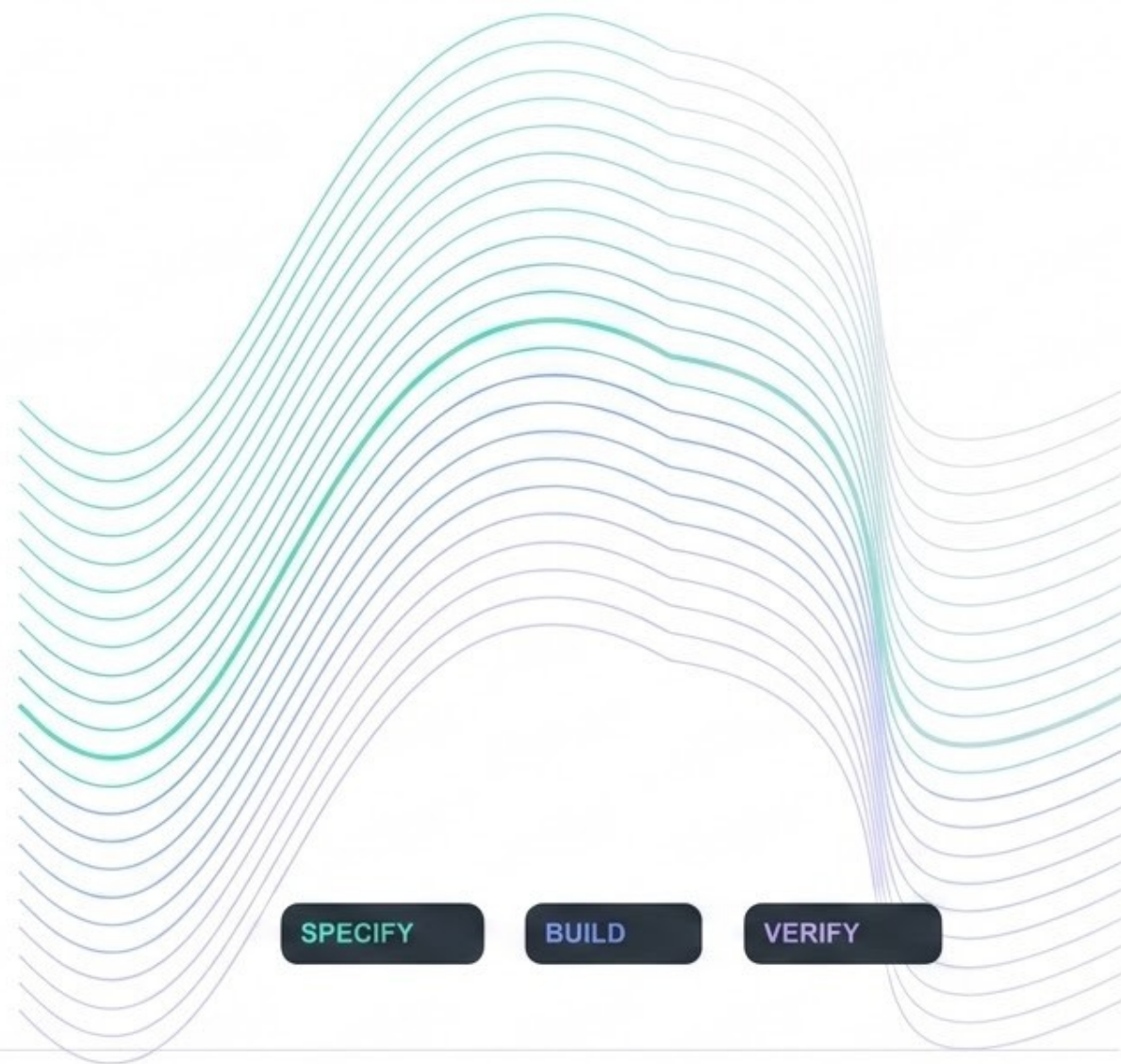


Riding the **AI wave**

as a full-stack engineer

From better prompts to maintainable,
well-tested software.



[Ibraheem Kabir] · [Technical Lead / Prune Payments]

[Conf 42]

01 / 15

Find the friction before the tool.

Start with a recurring bottleneck—not another subscription.

“Add a form that saves something.”



Where does the time actually go?

Delegate the work. Keep the judgment.

Treat generated code as a proposal, not proof of correctness.

DELEGATE

Drafts & exploration

- + Explain unfamiliar code
- + Propose implementations
- + Suggest edge cases

OWN

Decisions & evidence

- ✓ Confirm requirements
- ✓ Choose architectural fit
- ✓ Verify safety and readiness

Fast output still needs engineering evidence.

Choose tools by workflow.

Three demonstration roles—not three products you must subscribe to.



Claude Code

Read files, make changes
and run commands.

IN THIS TALK

Backend change



GitHub Copilot

Use editor assistance
to develop tests.

IN THIS TALK

Meaningful tests



Cursor

Work with codebase context
and inspect edits.

IN THIS TALK

Connected frontend

Overlapping capabilities. One reusable workflow.

One feature. End to end.

A task tracker gives every tool the same job to work towards.

THE ACCEPTANCE CRITERIA

Create a task.

- 01 Validate the title**
1–100 characters after trimming.
- 02 Save for the right user**
Use the authenticated owner.
- 03 Handle failure clearly**
Keep the input after an error.

The image shows a browser window with the address bar displaying 'localhost / tasks'. The main heading is 'My tasks'. Below it, there is a 'New task' section with a text input field containing 'Review AI-generated diff' and a green 'Add task' button. Below the input field, there is a list of tasks, with the first one being 'Write API tests' marked with a green checkmark. At the bottom of the window, there is a text label: 'ILLUSTRATIVE UI · REPLACE WITH YOUR APPLICATION'.

React + TypeScript



Node.js API



SQLite

Give the assistant a project brief.

Keep conventions close to the code, and keep them specific.

Claude Code

CLAUDE.md

GitHub Copilot

.github/copilot-instructions.md

Cursor

.cursor/rules/

working-agreements.md

Working agreements

Follow existing route patterns.

Reuse UI components.

Ask before adding dependencies.

Keep changes small.

Run tests and type checks.

Report checks you could not run.

Instructions guide behaviour. Permissions enforce boundaries.

Prompt like a good engineering ticket.

Goal + context + constraints + acceptance criteria + verification.

GOAL

CONTEXT

CONSTRAINTS

ACCEPTANCE

VERIFICATION

VAGUE

Build a task API.

No scope. No definition of done.

BOUNDED

Inspect the existing task routes and validation.
Plan a create-task endpoint with a trimmed title of 1–100 characters and the authenticated owner.
Reuse dependencies. List the files and tests needed.
Do not edit files yet.

Plan first. Approve the scope. Then implement.

From requirement to backend change.

Delegate a bounded implementation—and stop to review the diff.

Inspect → **Plan** → **Approve** → **Implement** → **Verify**

REVIEW THE DIFF

```
● ● ● claude / task-tracker
```

```
$ claude  
> Add POST /tasks using existing patterns.
```

Plan

- 1 Validate and trim the title.
- 2 Derive owner from the session.
- 3 Persist and return the saved task.
- 4 Add integration tests.

01 Validation

Reject blank and overlong titles.

02 Ownership

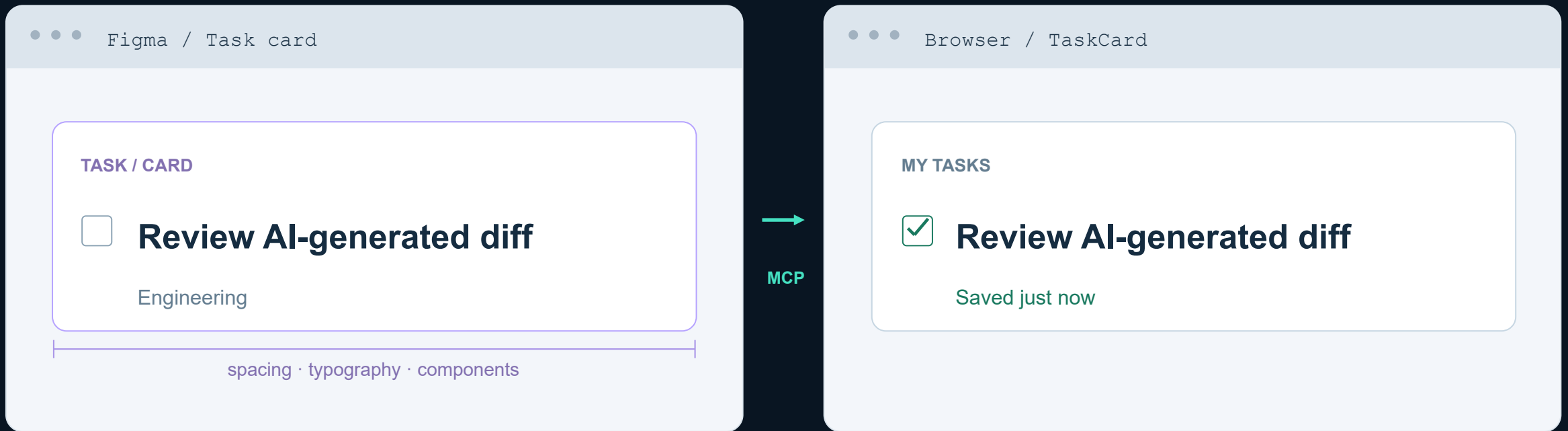
Use identity from the server.

03 Persistence

Check the saved record.

Use the design. Engineer the result.

A minimal Figma-to-code flow: design context, existing components, browser review.



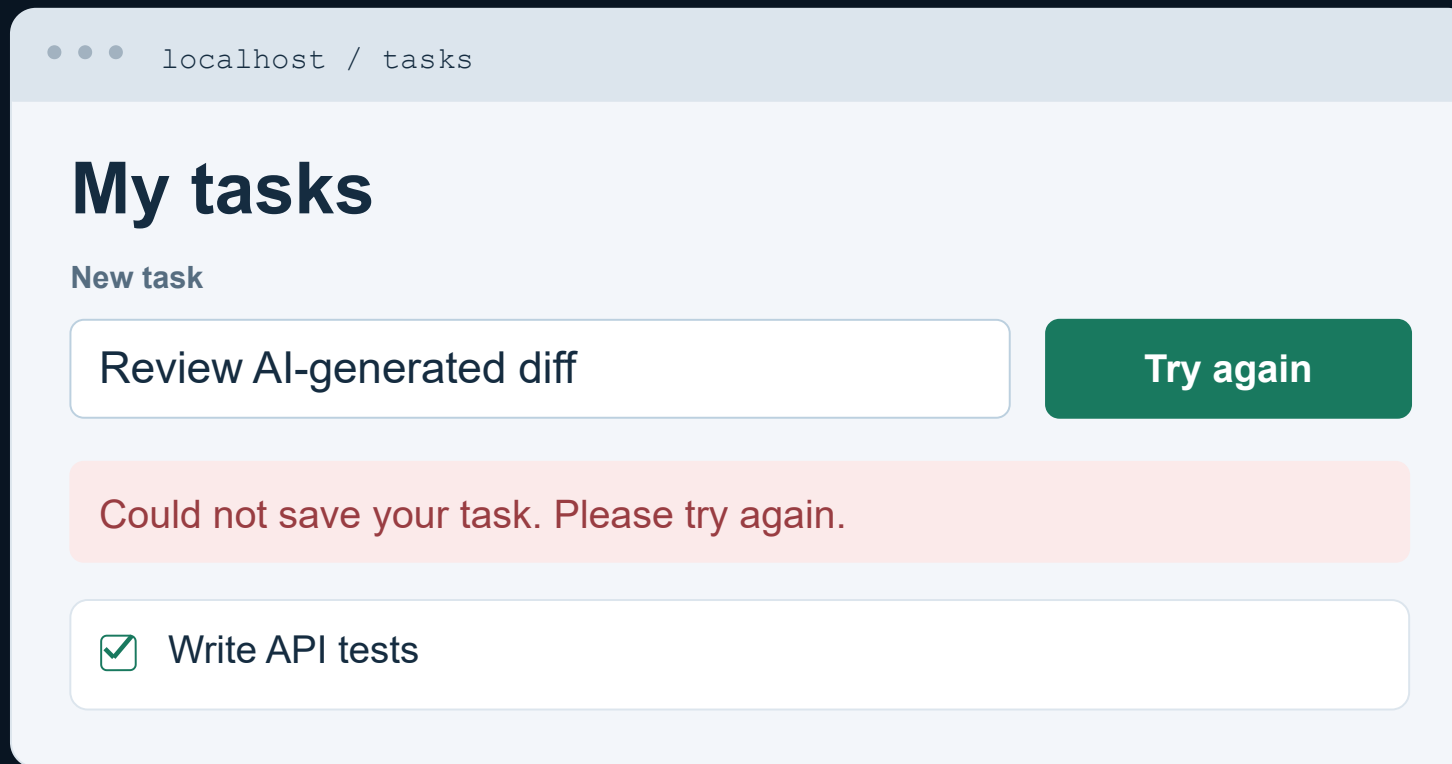
✓ Reuse design tokens

✓ Check keyboard access

✓ Test narrow screens

A working UI handles the unhappy path.

Connect the component to the real API contract—not just the happy-path mockup.



The screenshot shows a web browser window with the address bar displaying 'localhost / tasks'. The page title is 'My tasks'. Below the title, there is a section labeled 'New task'. Inside this section, there is a text input field containing the text 'Review AI-generated diff' and a green button labeled 'Try again'. Below the input field, there is a red error message: 'Could not save your task. Please try again.' At the bottom of the form, there is a checkbox labeled 'Write API tests' which is checked.

MAKE THE STATES EXPLICIT

- **Pending**
Disable repeat submissions.
- **Error**
Keep the user's input.
- **Success**
Render the saved task.

Demo: submit once, then simulate a failed request.

Tests should challenge the code.

Start from acceptance criteria, then ask whether a meaningful bug would be caught.

ACCEPTANCE → TEST CASES

- Valid task
- Whitespace-only title
- 100 / 101 characters
- Missing authentication

```
tasks.integration.test.ts

it("rejects blank titles", async () => {
  const res = await request(app)
    .post("/tasks")
    .set("Cookie", sessionCookie)
    .send({ title: "  " });

  expect(res.status).toBe(400);
});
```

Remove validation



Test must fail



Restore + re-run

Debug with evidence, not more guessing.

Reproduce → inspect → hypothesise → make the smallest fix → re-test.

Observed: the API fails, but the UI reports success.

BEFORE / seeded bug

```
const res = await fetch(
  "/tasks", options
);
setMessage("Task saved");
setTitle("");
```

AFTER / minimal fix

```
const res = await fetch(
  "/tasks", options
);
if (!res.ok) throw Error("Save failed");
setMessage("Task saved");
setTitle("");
```

Regression check: show the error and preserve the title.

Generated code. Same definition of done.

The change still needs evidence before it can ship.

01

Correctness

- ✓ Acceptance criteria
- ✓ Tests, types and build

02

Maintainability

- ✓ Existing patterns
- ✓ Small, readable diff

03

Safety

- ✓ Auth and data boundaries
- ✓ Limited permissions

REVIEWED PR

Verified checks + clear summary + human approval

Approve deployments and destructive actions explicitly.

Stay ahead without chasing every tool.

Try one workflow deliberately. Keep what earns its place.

2

WEEK EXPERIMENT

**One tool.
One recurring
task.**

01 **Baseline**

Record effort without assistance.

02 **Repeat**

Track prompting, review and rework.

03 **Compare**

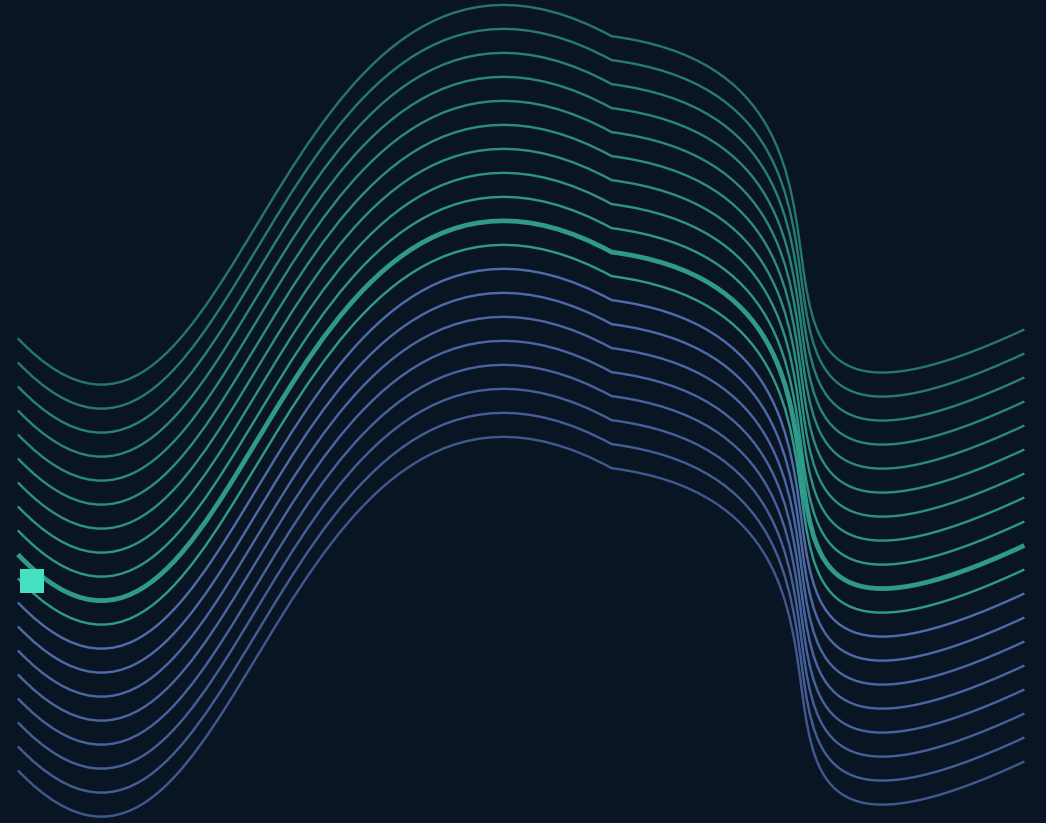
Judge the reviewed outcome—not generation speed.

Measure total effort. Keep what helps.

TAKE IT INTO YOUR NEXT PULL REQUEST

Ride the wave. Own what ships.

Ship software you understand,
can test, and can maintain.



CONTEXT

SMALL CHANGES

VERIFICATION

[Ibraheem kabir]

[ibraheem kabir / @deyemikabir]

